Design Rationale Req 3

Design 1:

1. Used downcasting and exception handling (try and catch) to identify and equip armor.
2. When the player attempted to wear an armor item, the program manually cast Actor to Wearing and Item to Armor.
3. This allowed WearAction to call specific armor method such as getDEFENSE() or getFunctionality() by forcefully treating the objects as their concrete types.
4. To prevent invalid casting, a custom exception GameRuleException was used to catch situations such as ClassCastException, when an actor tried to wear multiple armors or non-wearable items.

Pro	**Cons**
Simple to implement initially, work by only for small test scenarios	Violate OOP, especially encapsulation and abstraction
Direct access to concrete attributes through casting.	High connascence of type, dependent on exact class names and casting correctness.
	Violate OCP, new armor/equipment types required editing existing code
	High coupling, between WearAction, Actor, Armor, making the system poor.
	Exception handling replaces design structure, it reduce the clarity and may cause runtime risk to increased.

Although it work some how but it was relying on unsafe downcasting and exception control, which considered poor practice in modern OOP style.

Design 2

1. Introduce ArmorHolder ADT and injector interfaces (ArmorHolderInjector, WearActionInjector) to replace direct dependencies and casting.
2. Instead of actor directly managing its armor, they now hold a non portable ArmorHolder that stores the equipped armor.
3. Then WearAction now interacts with Wearing interface through dependency injection (DIP).

4. HandleArmorBlock statically manages block and healing effects after each successful attack, while ModifyIntrinsicWeapon, LootWeapon, ContinuousEffect integrates the armor system with combat behaviour.

Pro	**Cons**
Now downcasting anymore, replace with Wearing	More complex and more class due to additional layers.
Low connascence, class relate through interface not specific implementations	Need understanding on factory injector patterns.
High cohesion, each class performs single responsibilities	
Satisfies SOLID: SRP: Armor, ArmorHolder, HandleArmorBlock each handle distinct concerns. OCP: New armor type can be added without changes on exiting logic LSP: Subclasses (LeatherArmor, IronArmor, DiamondArmor) replace the based Armor. ISP: interfaces (Wearable, Wearing) expose only relevant methods. DIP: High-level modules depend on abstraction not concrete classes. KISS/DRY: -E.g., Common logic put in HandleArmorBlock prevents duplication such as healing back feature and validate against unconscious states.	
Extensible, support adding new effect into armor etc…	

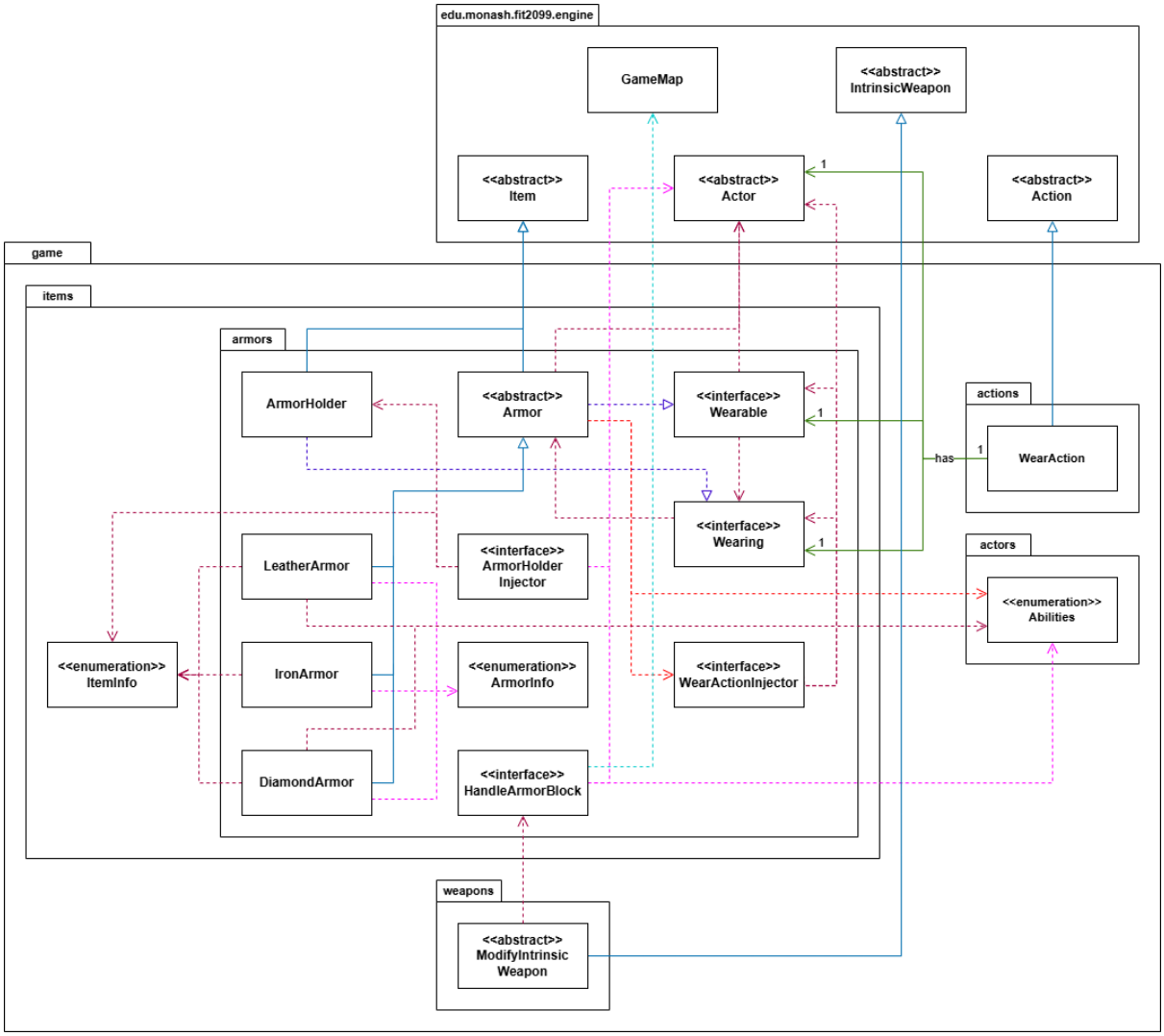
Chosen Design:

Design 2:

It completely removes unsafe casting and ensures OOP, SOLID principles. The separation of armor management (via ArmorHolder), combat modifications (ModifyIntrinsicWeapon, LootWeapon, ContinuousEffect), and post attack logic (HandleArmorBlock) ensuring low coupling, maintainability and extensibility.

This support future armor type with unique behaviors, such as elemental resistance or regeneration etc., without modifying existing logic.

Although the HandleArmorBlock logic could alternatively be placed within AttackAction class, but this decision is made to support future extensibility. For example, if a new weapon type is introduced that can ignore armor defense entirely (e.g., magic) a separate weapon class can simply omit the armor handling call. Satisfying OCP ensures that armor logic remains optional and independent from core attack.



Existing classes and responsibilities:

1. **Interface:**
    a. Wearing/Wearable → Defining armor equipping and information access behaviours.

   b. WearActionInjector/ArmorHolderInjector → Factory interface generate new instance object
   c. HandleArmorBlock → Static handler for block and healing back calculation post attack.
2. **Abstract:**
   a. Armor → Define common armor behaviour (defense, wearable actions).
   b. ModifyInstrinsicWeapon → Connects the armor system with attack flow to ensure consistent block behaviour.
3. **Concrete:**
   a. LeatherArmor, IronArmor, DiamondArmor → Armor types with unique defensive stats and ability.
   b. ArmorHolder → Store the current equipped armor for each actor (composition-based management)
   c. WearAction → Action to wear an armor.
4. **Enum:**
   a. ArmorInfo → Storing armor defensive value.

Classes Relate to and Interact with the Existing System

1. Actor possesses an ArmorHolder create through ArmorHolderInjector.
2. ArmorHolder manages the equipped armor and exposes block and armor info via Wearing.
3. When WearAction executes, it communicates only through interfaces
4. During combat, either in LootWeapon or ModifyIntrinsicWeapon will call HandleArmorBlock, which references the target's equipped armor.
5. Each armor subclass defines its own abilities using the Abilities enum (e.g., IMMUNE_STATUSES, COLD_RESISTANT) added into the owner.

Deliver Functionality

1. Actor who owns ArmorHolder which implement Wearing interface can wear an Armor.
2. When player chooses to wear an armor item on the ground, WearAction will removes the armor from the map and assign it to the actor ArmorHolder.
3. Each armor has different abilities based on type:
   a. Leather → block 2 damage and have cold resistance
   b. Iron → block 5 damage
   c. Diamond → Block 10 damage and has immunity to all status effects.
4. When attacked, the ModifyIntrinsicWeapon/LootWeapon triggers HandleArmorBlock.handleArmorBlock() after the base damage is applied.
5. If the armor's block value exceeds the incoming damage, the excess is converted to healing.
6. If the armor fails to block enough, the actor becomes unconscious.

7. The system allows only one armor per actor, preventing duplication or destruction.

Conclusion

Design 2 is clean, extensible and low coupling armor management system. ArmorHolder and injector interfaces help to eliminate the used of downcasting and tight dependencies while supporting modular extension. Each class follow SRP and integrate through well defined abstraction ensuring high cohesion and maintainability. The healing back also adheres to DRY and KISS, it also future scalability. So Design 2 is more suitable compared to design 1 as it satisfies both functional and non functional requirement through strong OOP, SOLID and low connascesne.